Crabbing Game (working title)

A solo-built, systems-driven commercial **fishing & crabbing simulation** in Unity — a living ocean you work for a living. Drive a boat out over a procedurally-built seabed, run crab pots, fight fish on a physically-modelled rod, then sort, store, and sell your catch under seasonal regulations, licenses, and community derbies.

Type: Solo personal project (ongoing) · **Role:** Sole designer, architect & developer · **Positioning:** Gameplay / Systems Engineering (primary) → Unity / Simulation → Game Design (secondary)

How to use this brief (for a résumé assistant): This is detailed source material on an in-progress game. Use it to write or tailor résumé bullets for a given role — select the most relevant details, keep all claims accurate, and don't overstate (it is an actively-developed prototype with deep, working systems, not a shipped product).

What it does

You take a boat out onto open water, find fish and crab where they actually live, and bring them in two different ways:

- **Crabbing** — bait and drop crab pots onto the seabed, mark them with buoys, motor away, and reel them back up later for whatever wandered in.
- **Fishing** — mount a rod at a station on the boat, cast a baited hook into the water, and **fight a hooked fish in real time** on a force-based rod-and-reel model (drag, reel, line tension, rod bend — all the real tackle controls).

The catch then flows through a full economy: a livewell on the boat, storage and gear management, a harbor where you dock and sell, **seasonal legality and weekly catch limits**, **licenses/ranks** that gate how much gear you can run and when you can fish, a **species journal** that unlocks knowledge as you catch, and **community fishing derbies** with shared quotas.

It's built to feel like a *job in a real ecosystem* — where the fish are depends on habitat, depth, season, and time of day, and how you land them depends on reading the fight.

The rod physics — a force-based tug-of-war in one shared currency

The centerpiece of the project. Rather than fake a fishing fight with a timing bar, the whole battle is a small **physics simulation where every actor speaks the same language: pounds (lbs).** Fish pull, reel drag, reel retrieve power, line tension, line breaking strength, and rod backbone are *all* measured in the same unit, so they balance against each other honestly and a designer tunes the fight by tuning forces, never magic numbers.

The one shared currency. A hooked fish pulls with a force in pounds. The component of that pull *along the line* is the load. Line tension = the drag-capped fish pull + your reel crank. The line snaps if tension holds above its breaking strength for ~0.5 s — so a steady pull alone can't break it, but cranking hard against a strong fish can. Who *gains line* is a separate arbitration: drag + reel-in power vs. the fish's pull. One global constant (`lbsToUnitsPerSec = 0.5`) converts net pounds into line speed for **every** piece of gear, so a pound of pull means the same thing across all rods and reels.

Drag discipline as emergent skill. The standout mechanic: **a fish's stamina burns with *resistance*, not time.** Stalled against a drag that holds it, it tires at full rate; freely stripping line off a light drag costs only a fraction of that. This reproduces real angling instinct — *work the fish against the brake to tire it; don't just wait.* The fish also only spends the force it *needs*: on a light drag it rips line at its top swim speed (not infinite force); against a strong brake it leans into its peak and gets held. Applied pull *eases* toward that target at a rate divided by the fish's rolled body weight, so **a heavy fish winds its heaves up and down slowly while a light one darts** — a shaped surge/rest sine with per-fish phase jitter makes each fish "work like a fish, not a metronome."

The rod bend is feedback, not the battle. The bend is computed from a quadratic spring (`spring =`

`rodMaxLoadLbs × bend²`) and folds fully as the load reaches the rod's backbone. It shapes how the rod *looks* under load and cushions tension spikes (a soft rod absorbs surges, a stiff rod transmits them) — but it deliberately does **not** decide who gains line, keeping the break-strength number true and portable across gear.

How the geometry is actually solved, each frame:

1. **Bend arc by integration.** An invisible helper carries the bend *direction*; the true rod tip is integrated along a quadratic-curvature arc from butt to tip (the curvature increases toward the tip, like a real blank loading up). The mesh-bend deformer and the fishing-line renderer both sample this same arc, so what you see matches the sim.
2. **Bend depth vs. bend plane, decoupled.** How *deep* the rod folds is the spring-vs-pull fight; which *direction* it leans is tracked nearly instantly toward wherever the fish actually is (a blank has no torsional resistance to its own bend plane). Splitting these fixed a subtle bug where, at equilibrium, the bend would freeze pointing the wrong way while the line pointed elsewhere.
3. **Fish-primary, line-tethered constraint.** The fish simulates *first* each frame (execution order) and the hook rides its mouth; the rod then reads how hard it pulls outward. The line-length sphere is **not a free teleport** when a fish is on — the player's heave is arbitrated in the same pounds, capped by what the blank can transmit over the fish's hold. The surplus loads the line (deeper bend, break timer, faster stamina burn), and an over-drag heave just slips the spool. That's the honest physics of leaning on a rod.
4. **Empty-hook pendulum.** With *no* fish, the hook is a Verlet-integrated spherical pendulum — gravity plus inertia, redirected tangentially by the line constraint — so casting and jigging swing naturally from the moving rod tip for free.

The control scheme is diegetic. The rod stays mounted at its station (never teleported to the hands); a dedicated station camera glides into the vantage and rod control is withheld until it settles. Mouse aims the rod's own pivot within a cone; LMB reels, RMB free-spools/pays out line (and is itself an escape risk if you give a fish slack), scroll sets drag 0–10. Two touches sell the feel: **aim gets heavier as the line loads** (a rod near breaking "barely answers the mouse"), and the rod **auto-rolls so its line guides face the load**, exploiting Unity's ZXY euler order so the guide twist is a clean final roll about the aimed blank.

Every rod and reel stat is read live from a swappable **`PoleData`** / **`ReelData`** **ScriptableObject** — there's a single source of truth, so the numbers physically *cannot* diverge between the asset and the runtime, and swapping gear is just swapping the asset.

How the world is built — one depth abstraction, everything hangs off it

The world is engineered around a **single spatial spine**: a `WaterLevel` singleton whose transform Y *is* the waterline, with one constant — `FeetPerUnit = 3` — defining the entire feet convention. All gameplay math stays in world units; depth is only ever *shown* in feet. Every other system reads depth from this one place instead of hardcoding heights, and a pluggable height sampler lets a wave provider feed displaced surface heights when present.

Procedural seafloor, built relative to the waterline (custom editor tool). A `SeafloorBuilder` `EditorWindow` (Tools ▸ Seafloor Builder) bakes a Unity Terrain from a *profile* and a *depth range in feet* — no hand-brushing. It uses **domain-warped Perlin noise** for "wiggly, fjord-like coasts," lifts the field toward the map edges so borders read as surrounding land, and **auto-paints sand/rock/grass splat maps by depth and slope** (rock on steep faces, grass above the beach band, sand on the seabed). It's non-destructive — it can re-shape an existing terrain, keep its footprint, reposition Y so the waterline maps to depth zero, and registers full Undo. The designer only ever thinks "the floor is N feet deep here."

Habitats, features, and data-driven species. Where life lives is declared, not coded:

- **`FishZone`** — a box-trigger habitat volume typed by a `Habitat` enum (Reef, OpenWater, Flats, Channel); a static registry the spawner iterates.
- **`FishSpot`** — hand-placed typed features inside a zone (Wreck, DropOff, WeedBed, SandBar…) that bias where schools drift, without being hardcoded "hot spots."

- **`SpeciesData`** (ScriptableObject) — *one asset per species*, no code per fish. It couples habitat preferences, a **depth band** (feet), **schooling model** (school size, member spacing, vertical band thickness, cohesion, solitary vs. schooling), **season heat** (per-season 0–10), **time-of-day activity** (peak hour + active window), bait affinity, and economy/legality into one declarative file.

Spawning reconciles all three, every in-game day. A `SeaCreatureSpawner` re-seeds the ocean on each day-advance: for every zone, it spawns the species that like that habitat, scales the school count by season heat (off-season *thins* schools but never zeroes them), and **raycasts the terrain to place each school at a valid depth** within the species' band — rejecting dry land and out-of-range depth. School footprint radius is *derived* from member spacing (`spacing × √count`) so **density stays consistent instead of being randomly rolled**, and a separation rule keeps schools apart. Crucially, a reseed never yanks a school out from under an active fight.

Schooling and streaming. A `SeaCreatureRoamArea` is the single source of truth for a school (position, quality, count, depletion) bound to its zone. Real fish agents are **pooled and streamed in only near the player** (within ~120 units, capped at ~12 visible fish regardless of school size), orbiting the school center and biasing toward liked features ~⅔ of the time. Performance care is baked in: seabed raycasts are **cached and only re-sampled after a school drifts ~0.5 units** ("the floor doesn't move, so a per-frame ray was pure overhead"), and swim animations are desynced with per-fish phase and speed jitter so a streamed school never paddles in unison. The same rolled "star quality" that scales a fish's pull and size is the quality you keep — **the fish you fight is the fish you keep.**

Time, seasons, and weather. A `TimeManager` runs a configurable day (≈20 real minutes), a 28-day-per-season / 4-season calendar, and per-season weather probability tables (Clear → Storm/Fog). Its `OnDayAdvanced` / `OnWeekAdvanced` events drive the daily ocean reseed, weekly catch-limit resets, license renewal fines, and seasonal derby resets — so the ecology, economy, and regulation layers stay in lockstep through one clock without referencing each other.

Boat handling, cleanly decoupled. The `BoatController` owns only XZ translation and heading on a gear-based throttle (reverse → stop → 5 forward gears), with speed-scaled steering (no turning at a dead stop) and a coast-to-stop engine cut. Vertical bob, pitch, and roll are left to the water-alignment component, so wave motion and player steering compose without fighting. All of it is tuned from a `BoatData` ScriptableObject, with a boat-upgrade system (speed, fuel, livewell, gear/bait/cooler slots) gated by the economy.

The systems around the catch

- **Progression** — tiered **licenses/ranks** (ScriptableObject-driven) that gate trap count and a daily fishing window, with a 30-day renewal cycle that fines you for lapsing; plus a separate level/XP track earned per item sold, cleanly splitting *economic* progression (money → rank) from *skill* progression (sales → level).
- **Regulation** — per-species **weekly sell limits** and **seasonal legality** (wrap-around season-date math), enforced at the point of sale and auto-reset on the weekly clock.
- **Derbies** — community fishing tournaments with a *shared pounds quota*: while active, a species can be sold without counting against personal limits, until the community pool fills and normal rules snap back.
- **Species journal** — a discovery system that unlocks one seasonal depth band at a time, tracks lifetime catches and personal bests, and surfaces a "new info" indicator.
- **Inventory & harbor** — dual-mode storage (infinite home inventory vs. constrained, stack-aware boat storage that mirrors real gear), a per-harbor shop, and a docking/undocking camera transition supporting a multi-boat fleet.
- **Other AI** — AI crabbers populate the water, and a per-fish bait-interest model (affinity × eagerness × closeness × time-of-day activity) decides whether a fish commits to your hook.

Key technical accomplishments

- **Designed a unified force model** for the entire fishing fight, with pounds as a single shared currency across fish

pull, drag, reel power, line tension, break strength, and rod backbone — so the fight is tuned by physics, not opaque difficulty constants.

- **Built a real-time rod simulation**: an integrated quadratic bend arc (shared by the mesh deformer and line renderer), decoupled bend-depth-vs-bend-plane solving, a fish-primary line-tether constraint with force-arbitrated player heave, and a Verlet pendulum for the free hook.
- **Made fish stamina a function of resistance, not time**, producing emergent "drag discipline" skill, and gave fish weight-based pull inertia, surge rhythm, and seabed-aware steering (dive in deep water, run along — not into — the bottom in the shallows).
- **Engineered a procedural seafloor editor tool** (domain-warped noise, depth-in-feet authoring, automatic depth/slope splat-mapping, non-destructive re-shaping with Undo).
- **Architected a data-driven ecology**: ScriptableObject species couple habitat, depth band, schooling, season, and time-of-day; a daily spawner reconciles habitat × species × depth via terrain raycasts with *derived* (not random) school density; schools pool and stream around the player with cached floor sampling for performance.
- **Tied ecology, economy, and regulation through one time system** (seasons, weekly limits, license renewals, derby resets) using event-driven, loosely-coupled manager singletons.

Tech stack

Language / **Engine:** C#, Unity 6 (URP 17.3) · **Rendering** / **Content:** Universal Render Pipeline, ProBuilder, Shader Graph, TextMeshPro, Unity Terrain + Terrain Tools · **Systems:** new Input System, Animation Rigging, Splines, Post-Processing · **Architecture:** ScriptableObject data-driven design, generic singleton managers, C# event bus · **Tooling:** custom `EditorWindow` tools (procedural seafloor builder, in-editor debug/tuning panels), Git · **Scale:** ~110 C# scripts, ~21k lines across fishing, crabbing, AI, world, progression, economy, UI, and camera systems.

Skills demonstrated

Gameplay / systems programming · real-time physics simulation · computational geometry / 3D math · emergent-mechanics design · Unity engine (C#) · ScriptableObject data-driven architecture · procedural generation · custom editor tooling (EditorWindow) · object pooling & streaming for performance · event-driven decoupling · singleton/manager architecture · AI behavior (state machines, steering, schooling) · economy & progression design · the new Input System · camera systems · clean separation of concerns · single-source-of-truth design · Git

---

Scope & honest status

- **Working, deep, and ongoing.** The fishing fight, crabbing loop, procedural world, spawning ecology, time/season system, boat handling, progression, regulation, derbies, journal, storage, and harbor are all implemented and interconnected — not mockups.
- **Actively in development.** Some areas are mid-rebuild by design (e.g., the inventory system is slated for a from-scratch pass), and there is **not yet a full save/load serialization layer** — persistence is partly asset-backed and partly in-memory. Don't claim a shipped product or a complete save system.
- Best framed as **a substantial solo-built simulation prototype** that demonstrates depth in gameplay-systems engineering, physics modelling, and data-driven Unity architecture — strongest as evidence of system design and engineering judgment, not as a finished commercial title.

Ready-to-use résumé bullets

- **Designed and built the core fishing combat of a Unity simulation game as a real-time force model** — fish pull, reel drag, line tension, break strength, and rod backbone all share one unit (pounds), so the fight balances on honest physics and is tuned without difficulty hacks.
- **Engineered a real-time rod-bend simulation** (integrated quadratic bend arc shared by mesh and line renderers, decoupled bend depth/plane, a fish-primary line-tether constraint with force-arbitrated player input, and a Verlet pendulum hook), plus a diegetic rod control scheme with load-weighted aim.
- **Created emergent angling skill by modelling fish stamina as a function of resistance rather than time**, with weight-based pull inertia and surge rhythm — reproducing real "work it against the drag" technique as a learned mechanic.
- **Architected a data-driven marine ecology** in which ScriptableObject species couple habitat, depth band, schooling, season, and time-of-day; a daily spawner reconciles habitat × species × depth via terrain raycasts with derived (consistent) school density; schools pool and stream around the player for performance.
- **Built a custom procedural-seafloor editor tool** (domain-warped Perlin noise, depth-in-feet authoring, automatic depth/slope texture splatting, non-destructive re-shaping with Undo) on a single waterline-relative depth abstraction shared by every gameplay system.
- **Integrated economy, progression, and regulation through one event-driven time system** — seasons, weekly catch limits, license renewals, and community derby quotas — using loosely coupled manager singletons and ScriptableObject-tunable content.

Project brief — Crabbing Game. Source material for résumé generation; all claims accurate to an actively-developed solo prototype.